AUtoMLy — Current Evidence Replay

October 16, 2026 · J.O.

This notebook recomputes observed historical-data audits, training summaries and development advisory counts from per-record, per-epoch and per-output inputs. It does not rerun training or inference. No final benchmark has been created or evaluated. Missing final and human-primary results remain missing, not zero. Local source-only recovery is not a model evaluation.

Run from the repository root or notebook/; alternatively set INAI6986_ROOT to the repository. Python standard library analysis; ipykernel is needed to execute. Source PDFs and model weights are intentionally not downloaded.

Code cell 1

```
from pathlib import Path
import os, json, hashlib, collections, statistics, re, math
roots = [Path(os.environ['INAI6986_ROOT'])] if 'INAI6986_ROOT' in os.environ else [Path.cwd(), *Path.cwd().parents]
ROOT = next(p for p in roots if (p/'notebook/evidence_20261016/manifest.json').is_file())
B = ROOT/'notebook/evidence_20261016'
manifest = json.loads((B/'manifest.json').read_text())
for name, meta in manifest['files'].items():
    assert hashlib.sha256((B/name).read_bytes()).hexdigest() == meta['shipped_sha256'], name
def load(name): return json.loads((B/name).read_text())
print('Verified shipped input fingerprints:', len(manifest['files']))
print('Original private-source fingerprints and transformations remain in manifest.json.')
```

```
Verified shipped input fingerprints: 6
Original private-source fingerprints and transformations remain in manifest.json.
```

Historical Corpus And Original Control Membership

These audit projections retain historical answer strings for word-length and overlap calculations, but omit generated questions, source passages, endpoints and protected control keys. A source-file-disjoint split is not proof of semantic independence; repeated answers and exposed historical tests remain limitations.

Code cell 2

```
data = load('historical_corpus_audit.json')
controls = load('original_control_membership.json')
assert len(data)==1999 and len(controls)==30
splits=collections.Counter(x['split'] for x in data)
sources=collections.defaultdict(set)
for x in data: sources[(x['source_url'],x['source_sha256'])].add(x['split'])
assert splits == {'train':1684,'validation':124,'test':191}
assert all(len(v)==1 for v in sources.values())
assert all(sources[(x['source_url'],x['source_sha256'])]=={'test'} for x in controls)
groups=collections.defaultdict(list)
for x in data: groups[' '.join(re.findall(r'[a-z0-9]+',x['answer'].lower()))].append(x)
cross=[v for k,v in groups.items() if k and len({x['split'] for x in v})>1]
assert len(cross)==16 and sum(map(len,cross))==57
lengths=sorted(len(x['answer'].split()) for x in data)
print('Rows / split:',len(data),dict(splits))
print('Source documents per split:',{s:sum(v=={s} for v in sources.values()) for s in splits})
print('Answer words: median',statistics.median(lengths),'95th percentile (nearest
rank)',lengths[math.ceil(.95*len(lengths))-1])
print('Cross-split duplicate-answer groups / member rows:',len(cross),sum(map(len,cross)))
print('Original controls: 30/30 historical-test source documents; 0 train/validation. Human dispositions received; later
remedy application is not inferred.')
```

```
Rows / split: 1999 {'train': 1684, 'validation': 124, 'test': 191}
Source documents per split: {'train': 113, 'validation': 10, 'test': 13}
Answer words: median 18 95th percentile (nearest rank) 52
```

```
Cross-split duplicate-answer groups / member rows: 16 57
Original controls: 30/30 historical-test source documents; 0 train/validation. Human dispositions received; later remedy
application is not inferred.
```

Sixteen Completed Training Runs

The same 124 validation prompt IDs were used. Within-run validation loss selects checkpoints; tokenizer-specific next-token accuracy is not policy-answer accuracy. Current rows exclude superseded early September methods. The replay reads saved measurements; it does not authenticate currently loaded GPU weights.

Code cell 3

```
gm=load('gemma_muse_epochs.json'); arch=load('architecture_epochs.json')
assert len(gm)==28 and len(arch['runs'])==8
assert sum(len(r['metrics_by_epoch']) for r in arch['runs'])==24
assert len({r['data_identity']['validation_prompt_ids_sha256'] for r in gm})==1
assert {r['data_identity']['validation_prompt_ids_sha256'] for r in gm}=={arch['shared_validation_prompt_ids_sha256']}
by_run=collections.defaultdict(list)
for r in gm:
    assert r['validation_rows']==124 and r['run']['test_rows_used']==r['run']['control_rows_used']==0
    by_run[r['run_id']].append(r)
assert len(by_run)==8
summaries=[]
for run,rs in by_run.items():
    final=max(rs,key=lambda r:r['step']); best=min((r for r in rs if r['validation']),key=lambda r:r['validation']['loss'])
    assert final['run']['status'].startswith('trainer_complete')
    summaries.append((final['model_id'].split('/')[-1],final['train_size'],final['step'],best['step'],best['validation']
['loss'],best['validation']['next_token_accuracy']))
for r in arch['runs']:
    assert r['status']=='trainer_complete' and r['test_rows_used']==r['control_rows_used']==0 and
r['remote_local_file_hashes_matched']==28
    best=min(r['metrics_by_epoch'],key=lambda e:e['eval_loss'])

summaries.append((r['model_family'],r['train_size'],r['global_step'],best['step'],best['eval_loss'],best['eval_token_accurac
y']))
assert len(summaries)==16
print('Current epoch rows: 28 Gemma/Muse + 24 Nemotron/Ornith; completed runs:',len(summaries))
print('N=1600 family | final step | selected step | selected loss | selected next-token accuracy')
for name,n,final,best,loss,acc in sorted(summaries):
    if n==1600: print(f'{name} | {final} | {best} | {loss:.4f} | {acc:.2%}')

Current epoch rows: 28 Gemma/Muse + 24 Nemotron/Ornith; completed runs: 16
N=1600 family | final step | selected step | selected loss | selected next-token accuracy
Muse-Glimmer-30B-unsloth-bnb-4bit | 600 | 200 | 0.5787 | 85.54%
gemma-4-26B-A4B-it | 600 | 400 | 0.5750 | 85.88%
nemotron | 1200 | 400 | 0.5527 | 85.56%
ornith | 1200 | 400 | 0.5626 | 85.38%
```

Development Advisory Matrix — Not Human Or Final Accuracy

The denominator is all 24 slots per condition, retaining generation/parser failures. Success is answered and correct and evidence-supported and no unsupported claims. This recomputes success from actual advisory dimensions, not a loaded aggregate. Error labels are mutually exclusive advisory categories, not exhaustive counts of every defect.

Code cell 4

```
out=load('development_outcomes.json')
assert len(out)==192 and len({(r['family'],r['condition'],r['prompt_id']) for r in out})==192
matrix=collections.defaultdict(collections.Counter); mix=collections.Counter()
for r in out:
    d=r['advisory_dimensions'] or {}
    success=(d.get('answer_outcome')=='Answered' and d.get('factual_correctness')=='Correct' and
d.get('evidence_support')=='Supported' and d.get('unsupported_claims')=='None')
    assert success == r['primary_pass'] and r['human_scored'] is False
    m=matrix[(r['family'],r['condition'])]; m['slots']+=1; m['successes']+=success
```

```
    m['judgments']+=r['judgment_present']; m['abstained']+=r['answer_outcome']=='Abstained'
    m['budget_failures']+=r['generation_failure_cause']=='terminal_model_budget_failure'
    m['parser_failures']+=r['generation_failure_cause']=='invalid_assistant_recipient_header'
    if d.get('error_category'): mix[d['error_category']]+=1
for key,m in sorted(matrix.items()): print(key,dict(m))
totals=collections.Counter()
for m in matrix.values(): totals.update(m)
assert dict(totals)=={'slots':192,'successes':17,'judgments':150,'abstained':48,'budget_failures':19,'parser_failures':23}
assert dict(mix)=={'Wrong Value':57,'Abstention Issue':32,'No Error':26,'Hallucinated Rule':26,'Invalid Locator':8,'Missing
Evidence':1}
print('TOTAL',dict(totals)); print('Error-label mix',dict(mix))
assert [matrix[(f,'base')]['successes'] for f in ['nemo','gemma','ornith','muse']]==[4,0,1,0]
assert [matrix[(f,'adapter')]['successes'] for f in ['nemo','gemma','ornith','muse']]==[4,4,4,0]
print('Three adapters tie at 4/24. No factual winner or deployment promotion.')
```

```
('gemma', 'adapter') {'slots': 24, 'successes': 4, 'judgments': 23, 'abstained': 0, 'budget_failures': 1, 'parser_failures':
0}
('gemma', 'base') {'slots': 24, 'successes': 0, 'judgments': 24, 'abstained': 14, 'budget_failures': 0, 'parser_failures':
0}
('muse', 'adapter') {'slots': 24, 'successes': 0, 'judgments': 0, 'abstained': 0, 'budget_failures': 1, 'parser_failures':
23}
('muse', 'base') {'slots': 24, 'successes': 0, 'judgments': 8, 'abstained': 8, 'budget_failures': 16, 'parser_failures': 0}
('nemo', 'adapter') {'slots': 24, 'successes': 4, 'judgments': 24, 'abstained': 0, 'budget_failures': 0, 'parser_failures':
0}
('nemo', 'base') {'slots': 24, 'successes': 4, 'judgments': 24, 'abstained': 3, 'budget_failures': 0, 'parser_failures': 0}
('ornith', 'adapter') {'slots': 24, 'successes': 4, 'judgments': 23, 'abstained': 0, 'budget_failures': 1,
'parser_failures': 0}
('ornith', 'base') {'slots': 24, 'successes': 1, 'judgments': 24, 'abstained': 23, 'budget_failures': 0, 'parser_failures':
0}
TOTAL {'slots': 192, 'successes': 17, 'judgments': 150, 'abstained': 48, 'budget_failures': 19, 'parser_failures': 23}
Error-label mix {'Wrong Value': 57, 'Abstention Issue': 32, 'No Error': 26, 'Hallucinated Rule': 26, 'Missing Evidence': 1,
'Invalid Locator': 8}
Three adapters tie at 4/24. No factual winner or deployment promotion.
```

Human Work, Recovery And Missing Final Evidence

Receipt counts below are metadata verification, not independently rescored human judgments. The planned final review uses visible machine advice and is therefore AI-assisted, not blind. Do not reuse original reviews as new output ratings.

Code cell 5

```
s=load('evidence_snapshot.json')
assert s['batch_a']['response_count']==27
assert (s['batch_b_rev11']['cards_reviewed'],s['batch_b_rev11']['ratings_count'],s['batch_b_rev11']
['revision'])==(30,180,11)
assert s['recovery_integration']['receipt_status']=='PASS_FOR_LUNA_EXECUTION_HANDOFF_NOT_FINAL_CAMPAIGN'
assert s['recovery_integration']['blocked_precondition'] is None
assert s['fresh_benchmark_created'] is False
print('Snapshot:',s['captured_shell_date_Central'])
print('Human work: Batch A 27 responses (9 supplementary pairs: 8 adopted AI drafts, 1 edited); Batch B 30 cards / 180
ratings, revision 11.')
print('Recovery: 226 copies preserved; 225 old keys retired/refused; 0 model calls. Not final inference.')
print('FINAL STATUS:',s['final_evaluation_status'])
for x in s['open_gates_before_final_inference']: print('Before inference:',x)
for x in s['open_gates_after_final_inference']: print('After inference:',x)
print('Additional protocol gap: reconcile committed human primary with full-cohort machine-advisory metric before sealing.
Planned human sample: 15 answerable + 3 unsupported; 36 paired outputs / 216 dimensions, not completed.')
print('Robustness: reviewed transform is first 60% of source characters, floor; same-metric evidence-surviving/erased
analysis remains UNMEASURED.')
final_human_accuracy=None
final_robustness_delta=None
print('Final human accuracy:',final_human_accuracy,'Final robustness delta:',final_robustness_delta)
```

```
Snapshot: 2026-10-16 15:03:32 CDT
Human work: Batch A 27 responses (9 supplementary pairs: 8 adopted AI drafts, 1 edited); Batch B 30 cards / 180 ratings,
revision 11.
Recovery: 226 copies preserved; 225 old keys retired/refused; 0 model calls. Not final inference.
```

```
FINAL STATUS: NOT RUN - final measurement gates remain open; documents built on this snapshot must not report final metrics
Before inference: Create representative local-AI benchmark and independently verify exact source/keys, including the flagged
pharmacy source exposure; source approval remains false
Before inference: Prove native runtime admission and exact matched Nemotron checkpoint-400/untouched-base assets and parity
Before inference: Seal exact accepted membership, keys, prompt, generation settings, scoring, human sample and robustness
perturbation before the one-time final run
After inference: Collect genuinely required human model-output ratings; original Batch A/B reviews do not cover new outputs
After inference: Recompute final metrics, same-metric robustness and subgroup results from actual rows; execute final
analysis notebook
Additional protocol gap: reconcile committed human primary with full-cohort machine-advisory metric before sealing. Planned
human sample: 15 answerable + 3 unsupported; 36 paired outputs / 216 dimensions, not completed.
Robustness: reviewed transform is first 60% of source characters, floor; same-metric evidence-surviving/erased analysis
remains UNMEASURED.
Final human accuracy: None Final robustness delta: None
```

AI Use And Accountability

Hermes Agent (Luna initial documents; GPT-6.1 Sol evidence review, replay code and final layouts) assisted reporting and deterministic analysis. Approved local models generated training/development outputs and machine advice. J.O. conceived and directed the project, chose models/data/metrics and stop/quarantine actions, supplied the recorded human work and retains responsibility. This offline replay does not fabricate human ratings, student authorship, current GPU identity or a final experiment.

Repository: https://github.com/porespellar/INAI6986_project_deliverables . Detailed source records remain private; instructor access and Canvas submission are separate unverified gates.